



PUSILKOM UI TRAINING

Directorate General of Taxes (DJP)
MLOps Planning and Design

Day 3: From Pipeline to Production

CI/CD/CT Design · Deployment Strategies · Monitoring & Observability · Incident Handling

Made Harta D., Tara Mazaya L., Naufal Mahdy H.

Copyright 2026 | Pusilkom UI · supported by GIZ



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



OVERVIEW

Day 3 Agenda

- | | | |
|----|--|-------------|
| 01 | CI/CD/CT Design for Machine Learning
MLflow model tracking & model registry, alias @champion | 08:30–10:30 |
| 02 | Machine Learning Deployment Strategies
Serving with FastAPI and Containerisation | 10:45–12:00 |
| 03 | Monitoring and Observability
Structured logging, data drift detection with Evidently AI, retraining policy | 13:15–14:45 |
| 04 | Orchestration and Incident Handling
orchestration with Prefect and incident response | 15:00–16:00 |

Where Day 3 Fits in the Programme

CONTEXT



Day 1

Foundations & governance

- DevOps vs MLOps
- Data privacy
- MLOps maturity assessment
- MLSecOps

Day 2

Data & reproducibility

- Data management & feature stores
- Data pipelines
- Reproducibility & versioning (DVC)
- Repositories, containers, registries

Day 3 — today

Operating models in production

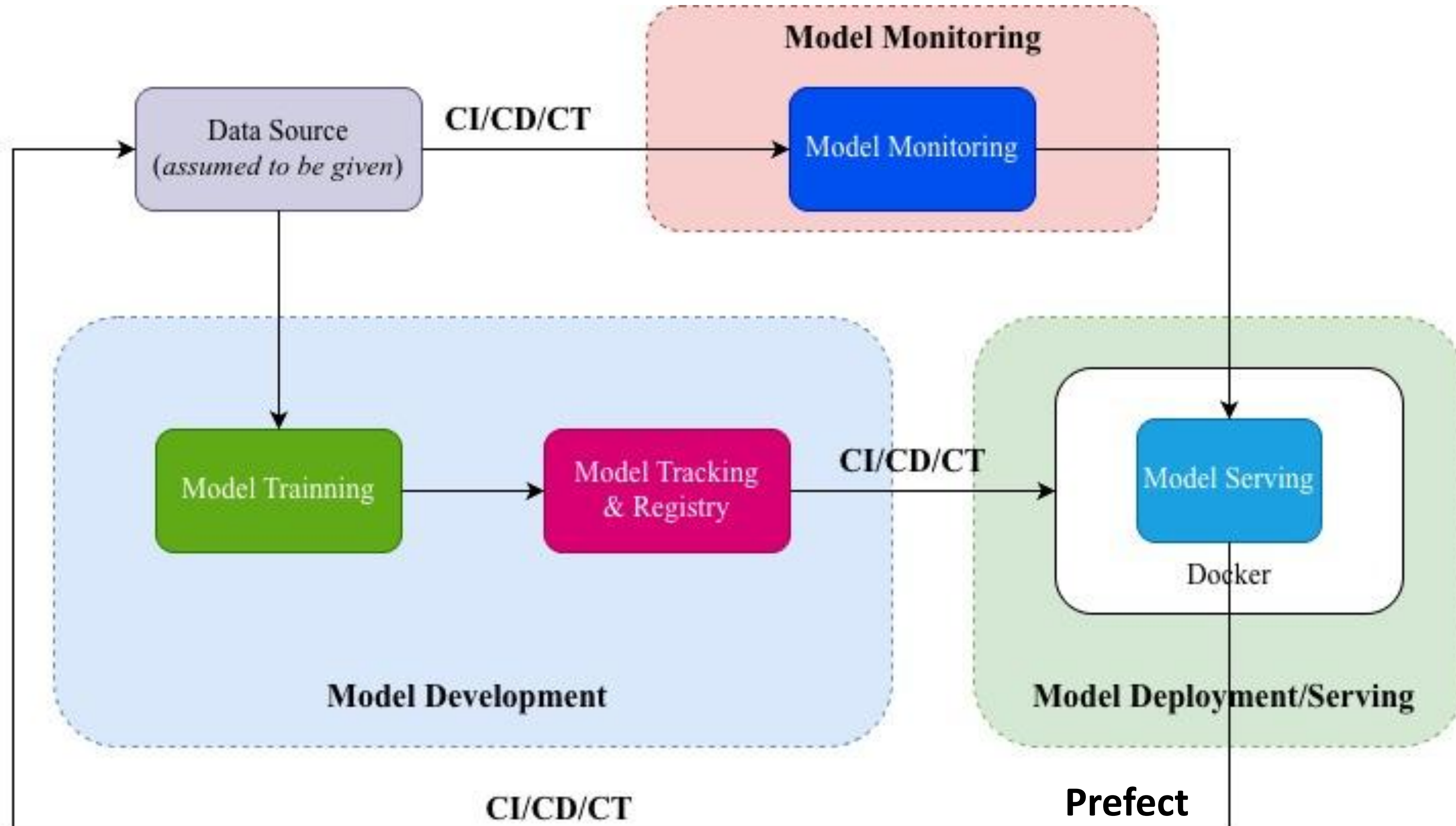
- CI/CD/CT
- Deployment strategies
- Monitoring & observability
- Incident handling

Our Today's Journey for MLOps

Topics



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



MLOps Tools for CI/CD/CT

Topics



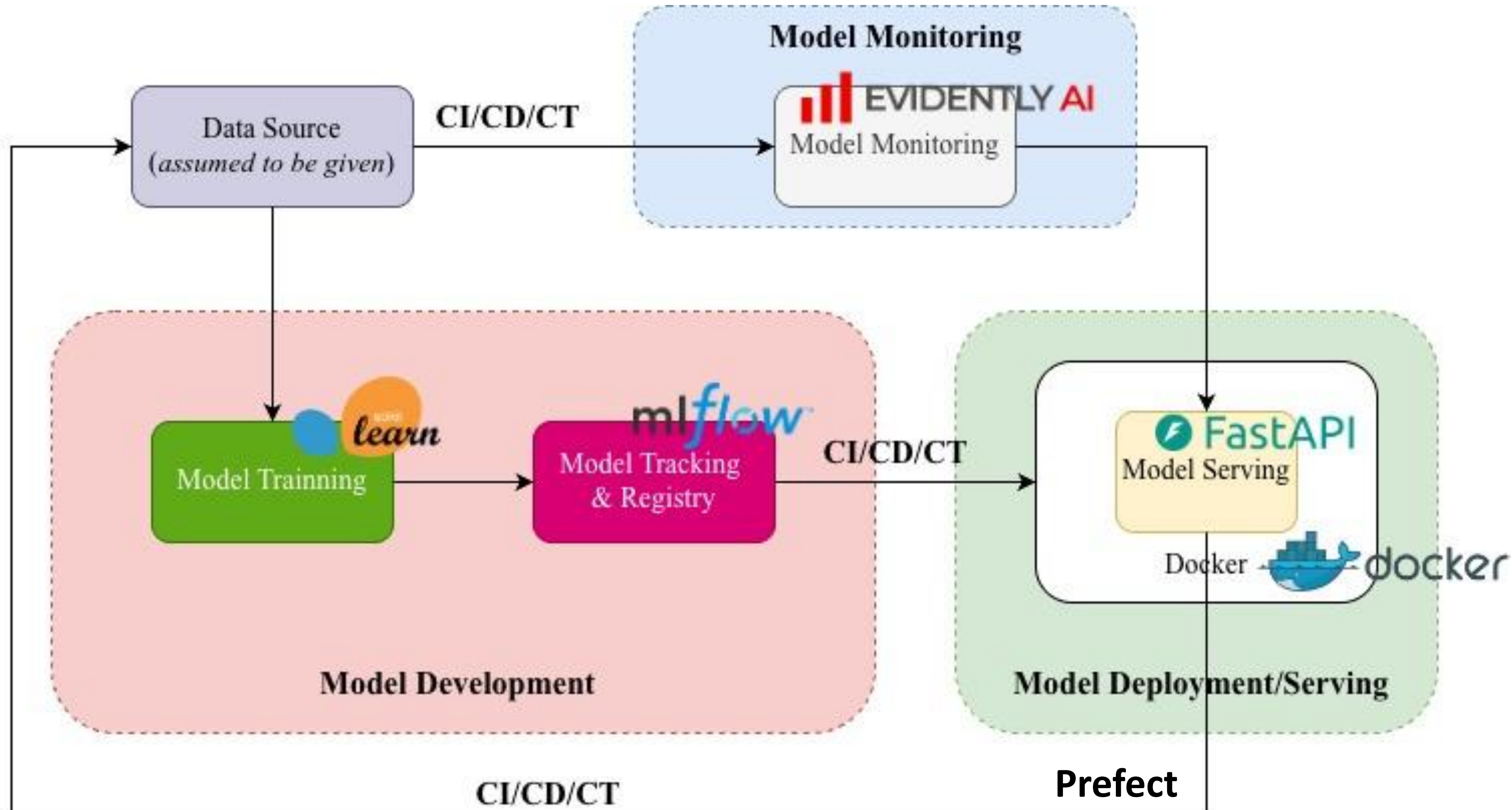
No	Phase	Tools
1	Model Development	Scikit-learn , XGBoost, TensorFlow, PyTorch, Keras, Spark Mlib, etc.
2	Model Tracking and Registry	MLFlow , Databricks, Clear ML, Comet, Kubeflow, Valohai, etc.
3	Model Deployment Env.	Docker Desktop , Google Cloud Run, AWS, etc.
4	Model Deployment	FAST API , Flask, BentoML, SageMaker, Kserve, etc.
5	Model Monitoring	Evidently AI , NannyML, Arize AI, Fiddler AI, WhyLabs, etc.
6	Orchestration	Prefect , ZenML, Airflow, Kubeflow, etc.

Our Today's Journey for MLOps

— Topics



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





Binary classification: approved (0/1)

File	Rows	Role
data/train.csv	1,500	Training
data/test.csv	500	Evaluation & monitoring reference
data/production.csv	500	'Production' data with deliberate drift

8 numeric features: age, income, bmi, tenure_months, num_claims, credit_score, policy_value, risk_score.

Source Code Structures

Topics

<https://github.com/madeharta/latest-mlops-workshop>

```
mlops-workshop/
├── requirements.txt          # dependency development lokal (root)
├── config.yml               # Environment test -- konfigurasi model
├── train.py                 # Environment test -- pipeline training baseline
├── Makefile                 # Environment test -- automation layer
├── data/
│   ├── generate_dataset.py  # cara dataset dibuat (opsional dijalankan ulang)
│   ├── train.csv / test.csv # data training & evaluasi
│   └── production.csv       # data "produksi" dengan drift disengaja (Sesi 7)
├── tests/test_pipeline.py   # Environment test -- unit test contoh
├── mlflow/
│   ├── train_with_mlflow.py # Tracking and Registry -- tracking + registry, bootstrap alias "production"
│   ├── export_model.py      # jembatan registry (alias production) -> file statis untuk Docker
│   ├── retrain_and_compare.py # CT: retrain + bandingkan + promosikan jika lebih baik
│   ├── zmlflow_explore_tracking.py # Advanced tracking exploration
│   ├── ztrain_with_mlflow_autologging.py # Autologging feature testing
│   └── ztrain_with_mlflow_regression.py # Autologging with regression model
├── api/
│   ├── app.py               # FastAPI serving
│   └── requirements.txt      # dependency ramping khusus image Docker
├── docker/
│   ├── Dockerfile           # Model deployment
│   ├── docker-compose.yml   # deployment lokal (pengganti Cloud Run)
│   └── local_deploy.sh       # CI/CD lokal: export + build + redeploy
├── monitoring/
│   ├── monitor.py           # Model monitoring: data drift + prediction drift + performance
│   ├── run_evidently_service.sh # Run evidently UI as docker container
│   ├── zmonitor_workspace.py # Code for testing the workspace so that the report is available on UI
│   └── requirements.txt      # Dependency, already provided in the root folder
├── scripts/test_deployment.py # Model serving: uji endpoint lokal yang sudah live
└── orchestration/
    ├── pipeline.py          # Otomisasi: CI/CD/CT dengan Prefect
    └── requirements.txt
```



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



The Prepared Repo & Environment

ENVIRONMENT

Already on every laptop / VM

- VSCode installed or any other editor
- mlops-workshop repo cloned from GitHub
- Python >3.10 with an active virtual environment
- Dependencies from requirements.txt (pinned)
- Prefect installed from orchestration/requirements.txt
- Train / test / production data already in data/
- Docker available (deployment session)

```
bash - terminal

# quick check (1 minute)
$ source venv/bin/activate
$ python --version
Python 3.11.x
$ python -c "import numpy as n; print(n.__version__)"
1.26.4      # must be < 2
$ make test
2 passed
$ make preflight    # PASS/WARN/FAIL
```



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



01

CI/CD/CT Design for Machine Learning

08:30–10:30

IN THIS SESSION

- 01 CI, CD and CT for ML
- 02 MLflow experiment tracking
- 03 Exploring MLflow features
- 04 Aliases instead of stages
- 05 MLflow model Registry
- 06 Exercise and Discussion

CI, CD and CT in Machine Learning

CONCEPT



CI

Continuous Integration

- Every code change is tested automatically
- In ML: also test the data and model quality
- Example: pytest + quality gate

CD

Continuous Delivery / Deployment

- Artefacts that pass tests are released automatically
- In ML: the artefact is an image + a specific model version
- Example: build image → Cloud Run

CT

Continuous Training

- The model is retrained when the data changes
- Triggered by a schedule, new data, or drift
- Example: monitor → retrain → registry

We will defer this section to the end of workshop after all topics are complete

The Three MLflow Components We Use

COMPONENTS

01 Tracking

Records the parameters, metrics and artefacts of every run:
log_param, log_metric, log_model.

02 Model Registry

A catalogue of models with version numbers and aliases (@champion). The basis for release and rollback.

03 MLflow UI

A dashboard to compare runs: sort, filter and plot metrics across experiments.



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Why Experiment Tracking Matters

PROBLEM



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



01 Lost version details

"Which model gave us 93% accuracy?" Without tracking, the honest answer is often "I don't remember".

02 Hard to compare

Notebook prints vanish when the kernel restarts. There is no systematic way to compare dozens of runs.

03 Collaboration clashes

Two people train similar models without knowing, duplicating work or overwriting each other's results.

04 No audit trail

A production model cannot be traced back to the code, data and parameters that produced it.

Tracking Backend: SQLite, Not a File Store

△ VERSION FINDING



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



ROOT CAUSE

The folder-based backend (file:///.../mlruns) is in maintenance mode in recent MLflow releases and may be rejected when `set_experiment()` is called.

FIX

Use a SQLite backend. It stays local (a single `mlflow.db` file), needs no separate database server, and is fully supported.

OR

<http://domain:port> is also working, but it requires MLFlow server already started



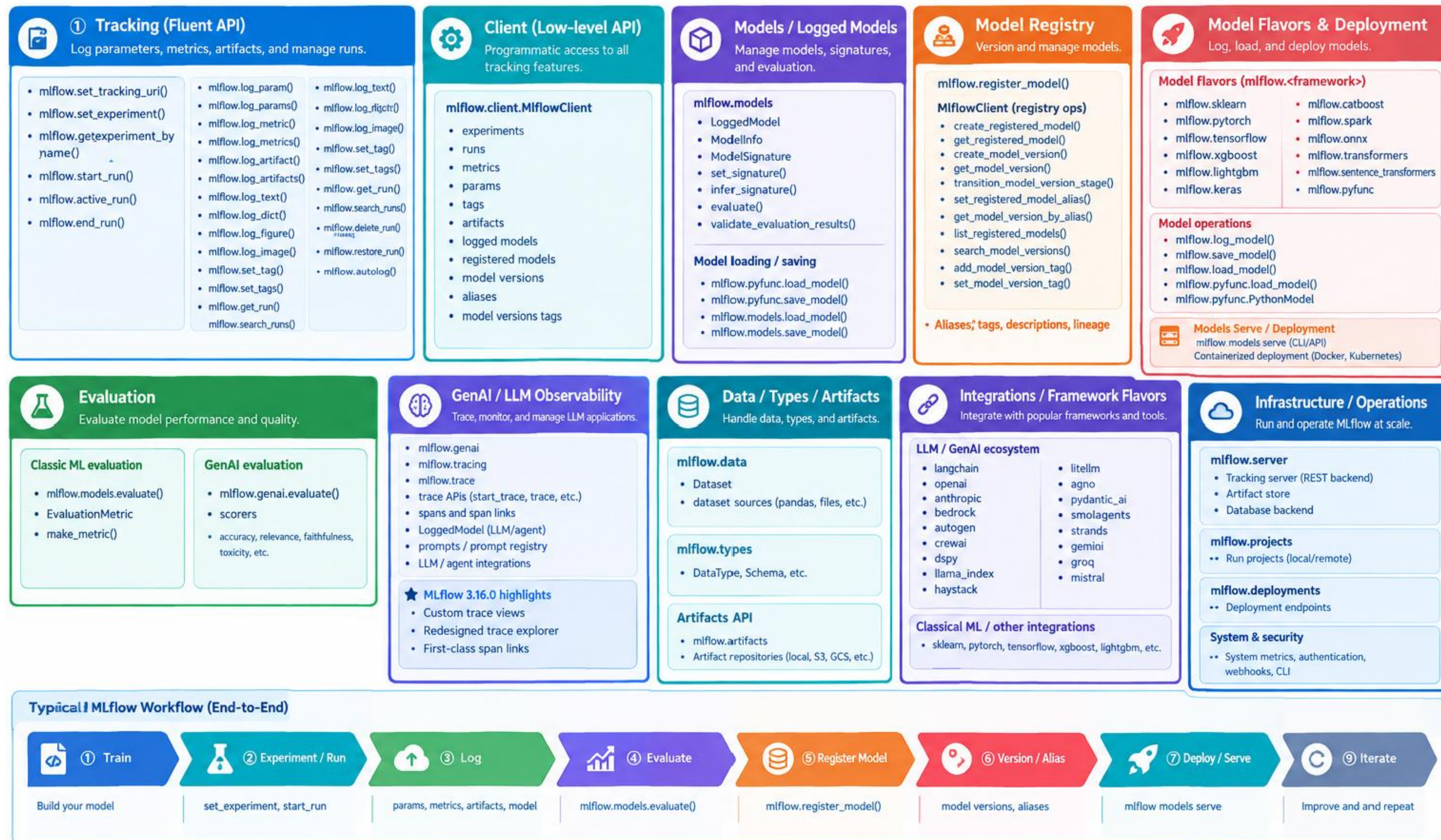
```
mlflow.set_tracking_uri("sqlite:///mlflow.db")  
# mlflow.set_tracking_uri("http://127.0.0.1:9001")  
# mlflow.set_tracking_uri("file:///.../mlruns")
```

```
# CORRECT  
# CORRECT → but requires MLFLOW server running  
# avoid
```

MLflow API Structure in Detail



VERSION FINDING

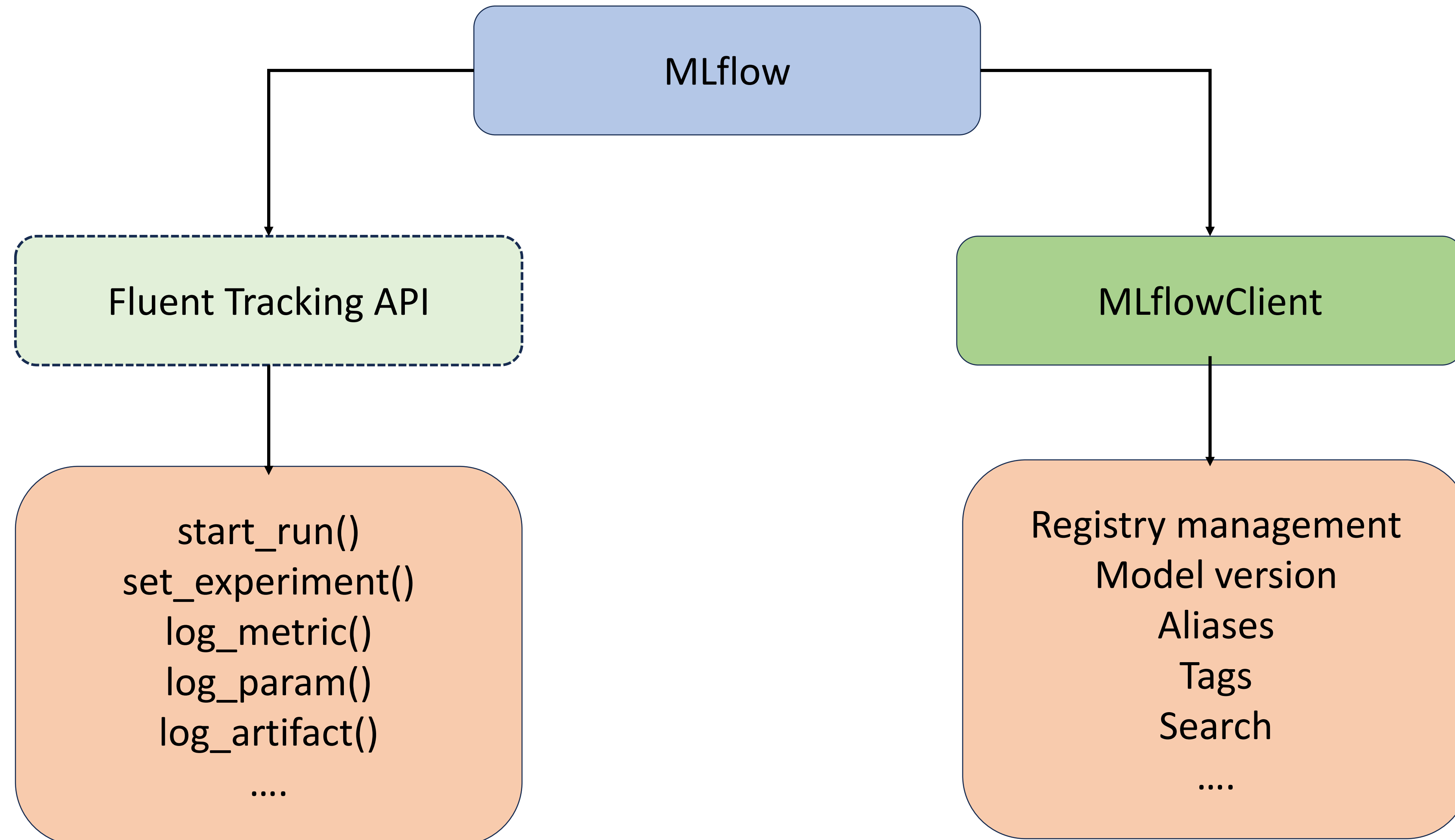


MLflow API : Mlflow vs MLflowClient

△ VERSION FINDING



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



A Note about Model Format

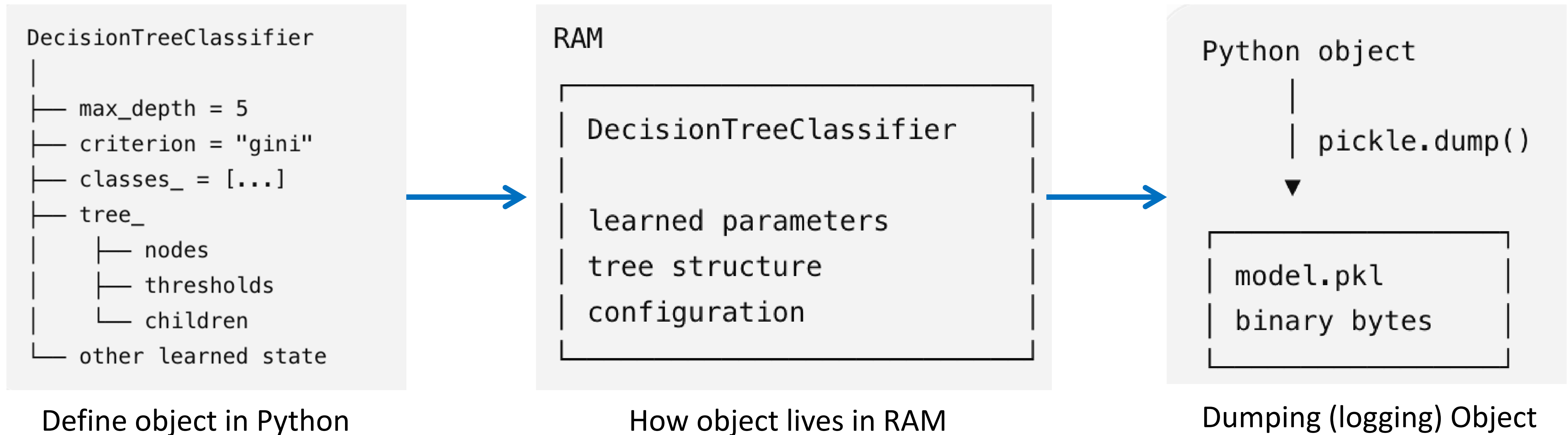
VERSION FINDING



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



In this workshop, we mainly use python with scikit-learn framework for model development.



- Scikit-Learn : Pickle / joblib / skops → .pkl, .joblib, .skops
- Xboost : JSON / UBJSON → .json, .ubj
- TensorFlow : SavedModel → directory
- Keras : Keras v3 format → .kreas
- PyTorch : state_dict / PyTorch serialization → .pt, .pth

Run Three Experiments at Once

HANDS-ON 1



● ● ● bash – terminal (real output)

```
$ make mlflow
```

```
DecisionTree: acc=0.8740 auc=0.9075 f1=0.8706
```

```
RandomForest: acc=0.9380 auc=0.9817 f1=0.9350
```

```
GradientBoosting: acc=0.9380 auc=0.9804 f1=0.9347
```

```
BEST: RandomForest (accuracy=0.9380)
```

```
Registered version: 1
```

```
Alias 'champion' -> version 1
```

● ● ●

```
$ make ui      # mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001
```

MLflow UI — Comparing Runs

REAL SCREENSHOT



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



mlflow
3.16.0

GenAI Model training

← Insurance Approv...

Runs

Models

Traces

Model registry

Experiments > Insurance Approval Model >

Runs ⓘ

metrics.rmse < 1 and params.model = "tree"

All time

State: Active

Datasets

Sort: Created

Columns

Group by

							Metrics			
	Run Name	Created	Dataset	Duration	Source	Models	accuracy	f1_score	roc_auc	
☐	GradientBoosting	✓ 1 minute ago	-	10.9s	train_wi...	model	0.938	0.93473684...	0.98035668...	⊕
☐	RandomForest	✓ 1 minute ago	-	6.3s	train_wi...	insurance-approval-model...	0.938	0.93501048...	0.98174948...	
☐	DecisionTree	✓ 1 minute ago	-	8.2s	train_wi...	model	0.874	0.87063655...	0.90750672...	

Model training toggle

The UI opens in GenAI mode by default.
Switch to Model training for classic runs.

Columns → metrics

Add accuracy, f1_score and roc_auc, then
click a header to sort.

Models column

The RandomForest run links to insurance-
approval-model in the registry.

Instrumenting Training with MLflow

CODE

Experiments Tracking: manual logging (train_with_mlflow.py)

```
mlflow/train_with_mlflow.py

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("Insurance Approval Model")

for name, (ModelClass, params) in MODELS.items(): # DecisionTree, RandomForest, GradientBoosting
    with mlflow.start_run(run_name=name) as run:
        model = ModelClass(**params, random_state=42)
        model.fit(X_train, y_train)

        preds = model.predict(X_test)
        proba = model.predict_proba(X_test)[:, 1]]

        for k, v in params.items():
            mlflow.log_param(k, v)
        mlflow.log_metric("accuracy", accuracy_score(y_test, preds))
        mlflow.log_metric("roc_auc", roc_auc_score(y_test, proba))
        mlflow.log_metric("f1_score", f1_score(y_test, preds))
        model_info = mlflow.sklearn.log_model(model, name="model")

    ...
```



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Instrumenting Training with MLflow

CODE

Experiments Tracking: manual logging

←

→

↺

🏠

🔔

127.0.0.1:9001/#/experiments/1/runs?searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&m

mlflow

3.16.0

GenAI

Model training

←

🔔

Insurance Appro...

☰

Runs

🔗

Models

🔗

Traces

🔗

Model registry

☰

Runs

🔔

☰

📈

📊

🔍 metrics.rmse < 1 and params.model = "tree" 🔔

🕒 All time ▾

State: Active ▾

Datasets ▾

📄 Columns ▾

☐ Expand rows

📄 Group by ▾

Metrics						
☐	Run Name	Created ↕	Duration	accuracy	f1_score	roc_auc
☐	● GradientBoosting	✅ 16 minutes ago	2.0s	0.938	0.9347368421052632	0.9803726946721311
☐	● RandomForest	✅ 16 minutes ago	2.1s	0.938	0.9350104821802935	0.9817494877049181
☐	● DecisionTree	✅ 16 minutes ago	4.5s	0.874	0.8706365503080082	0.9075067238729508
☐	🔍 retrain-challenger-...	✅ 15 hours ago	11.8s	-	-	-
☐	● GradientBoosting	✅ 15 hours ago	3.5s	0.938	0.9347368421052632	0.9803726946721311
☐	● RandomForest	✅ 15 hours ago	3.4s	0.938	0.9350104821802935	0.9817494877049181
☐	● DecisionTree	✅ 15 hours ago	4.9s	0.874	0.8706365503080082	0.9075067238729508
☐	● DecisionTree	❌ 16 hours ago	3.8s	0.874	0.8706365503080082	0.9075067238729508

Autologging with MLflow

CODE

Experiments Tracking: auto-logging (train_with_mlflow_autologging.py)

```
mlflow/train_with_mlflow.py

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("Insurance Approval Model")

mlflow.sklearn.autolog(
    log_input_examples=True,
    log_model_signatures=True,
    log_models=True
)

for name, (ModelClass, params) in MODELS.items(): # DecisionTree, RandomForest, GradientBoosting
    with mlflow.start_run(run_name=name) as run:
        model = ModelClass(**params, random_state=42)
        model.fit(X_train, y_train)

        preds = model.predict(X_test)
        proba = model.predict_proba(X_test)[:, 1]
        mlflow.log_metric("accuracy", accuracy_score(y_test, preds))
        mlflow.log_metric("roc_auc", roc_auc_score(y_test, proba))
        mlflow.log_metric("f1_score", f1_score(y_test, preds))

    ...
```

MLFlow is designed to observe ML model training, thus information from prediction is not included in auto-logging information and must be manually logged



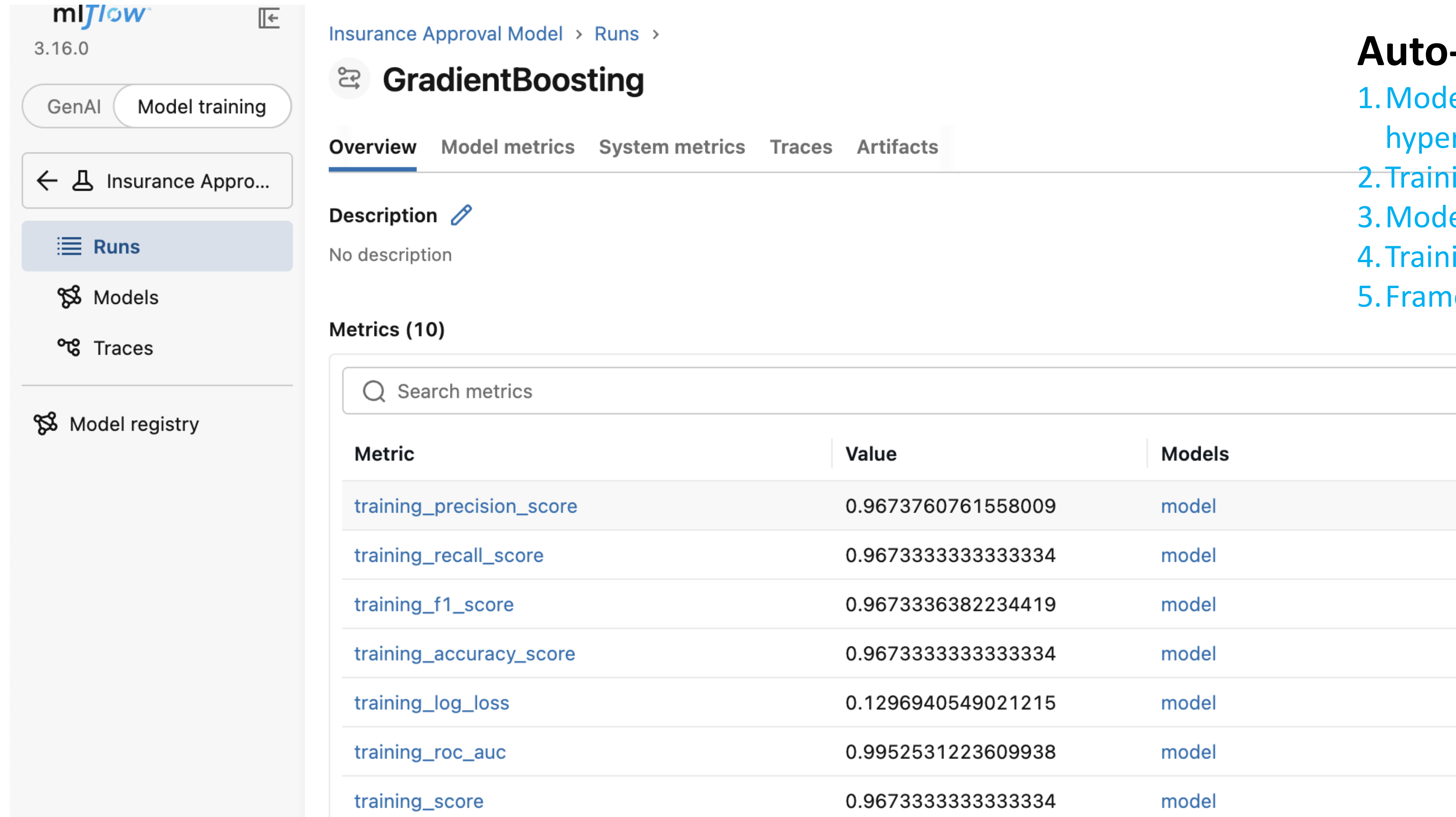
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Autologging with MLflow

CODE

Experiments Tracking: auto-logging (train_with_mlflow_autologging.py)



The screenshot shows the MLflow web interface for an experiment named 'Insurance Approval Model'. The left sidebar contains navigation links for 'GenAI', 'Model training', 'Runs', 'Models', 'Traces', and 'Model registry'. The main content area shows the 'GradientBoosting' run with tabs for 'Overview', 'Model metrics', 'System metrics', 'Traces', and 'Artifacts'. The 'Overview' tab is selected, showing a description field with 'No description' and a 'Metrics (10)' section. The metrics table lists the following data:

Metric	Value	Models
training_precision_score	0.9673760761558009	model
training_recall_score	0.9673333333333334	model
training_f1_score	0.9673336382234419	model
training_accuracy_score	0.9673333333333334	model
training_log_loss	0.1296940549021215	model
training_roc_auc	0.9952531223609938	model
training_score	0.9673333333333334	model

Auto-Logging information

1. Model parameters and hyperparameters
2. Training and validation metrics
3. Model artifacts and checkpoints
4. Training plots and visualizations
5. Framework-specific metadata

Autologging with MLflow

CODE

Classification vs Regression Logging(ztrain_with_mlflow_regression.py)

```
25 from sklearn.linear_model import LinearRegression
26 from sklearn.tree import DecisionTreeRegressor
27 from sklearn.ensemble import RandomForestRegressor
28 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
29
30 #mlflow.set_tracking_uri("sqlite:///mlflow.db")
31 mlflow.set_tracking_uri("http://127.0.0.1:9001")
32 mlflow.set_experiment("Risk Score Regression")
33
```

```
45
46 MODELS = {
47     "LinearRegression": (LinearRegression, {}),
48     "DecisionTree": (DecisionTreeRegressor, {"max_depth": 6, "random_state": 42}),
49     "RandomForest": (RandomForestRegressor, {"n_estimators": 200, "max_depth": 8, "random_state": 42}),
50 }
51
```

```
60
61     mae = mean_absolute_error(y_test, preds)
62     mse = mean_squared_error(y_test, preds)
63     r2 = r2_score(y_test, preds)
64
```



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Autologging with MLflow

CODE

Classification vs Regression Logging(ztrain_with_mlflow_regression.py)



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



← → ↺ 🏠 ⓘ 127.0.0.1:9001/#/experiments/3/runs/aa948dbea75c4c7baf1dec2000a3ce22

mlflow

3.16.0

GenAI Model training

← 📁 Risk Score Regre...

☰ Runs

🔗 Models

🔗 Traces

🔗 Model registry

Risk Score Regression > Runs >

🔄 LinearRegression

Overview Model metrics System metrics Traces Artifacts

Description ✎
No description

Metrics (8)

🔍 Search metrics

Metric	Value	Models
training_mean_squared_error	35.99593199658499	model
training_mean_absolute_error	4.21732742269553	model
training_r2_score	0.9393252637040765	model
training_root_mean_squared_error	5.999660990138109	model
training_score	0.9393252637040765	model

Metrics yang di log disesuaikan secara otomatis oleh mekanisme autologging

Exploring Experiment Tracking with MLflow

CODE

<https://mlflow.org/docs/latest/ml/tracking/tracking-api/> (mlflow_explore_tracking.py)

←

→

↺

🏠

🔍

mlflow.org/docs/latest/ml/tracking/tracking-api/

mlflow

Machine Learning

API Reference

Self-Hosting

Community

Version: 3.16.1 (latest)

Overview

Getting Started

Machine Learning

Traditional ML

Deep Learning

Build

MLflow Tracking

Quickstart

Auto Logging

Tracking Server

Search

System Metrics

Tracking APIs

MLflow Model

ML flow Datasets

Artifact Management

Function	Purpose	Example
<code>mlflow.log_artifact()</code>	Log single file/directory	<code>mlflow.log_artifact("model.pkl")</code>
<code>mlflow.log_artifacts()</code>	Log entire directory	<code>mlflow.log_artifacts("./plots/")</code>
<code>mlflow.get_artifact_uri()</code>	Get artifact storage location	<code>uri = mlflow.get_artifact_uri()</code>

Model Management (New in MLflow 3)

Function	Purpose	Example
<code>mlflow.initialize_logged_model()</code>	Initialize a logged model in PENDING state	<code>model = mlflow.initialize_logged_model(name="my_model")</code>
	Create	



Deutsche Gesellschaft für Internationale Zusammenarbeit (GIZ) GmbH



Exploring Experiment Tracking with MLflow

CODE

MLFlow Logging Results

←

→

↺

🏠

🔍 127.0.0.1:9001/#/experiments/2/runs/cfdfec9295964815ba3e18b13a33620d/artifacts

mlflow

3.16.0

GenAI

Model training

← 📁 Insurance Appro...

☰ Runs

🔗 Models

🔗 Traces

🔗 Model registry

Insurance Approval Tracking API > Runs >

🔗 GradientBoosting

Overview

Model metrics

System metrics

Traces

Artifacts

▼ 📁 evaluation

📄 GradientBoosting_confusion_matrix.png

📄 GradientBoosting_evaluation.json

▼ 📁 feature_importance

📄 GradientBoosting_feature_importance.c...

evaluation/GradientBoosting_confusion_matrix.png 18.01KB

Path: /Users/made/Documents/latest-mlops-workshop/mlruns/2/cfdfec9295964815ba3e18b13a33620d/artifac

Confusion Matrix - GradientBoosting

Actual

Rejected

Approved

Rejected

Approved

Predicted

247

9

22

222

200

150

100

50

11

A centralized model store, set of APIs, and UI for managing the full ML model lifecycle

Provides lineage, versioning, aliasing, and metadata tagging, so every model has full traceability from the experiment that produced it through to production deployment.

The importance of Model Registry

Version Control	Automatically tracks every model version — compare iterations, roll back, and manage staging vs. production in parallel.
Lineage & Traceability	Each version links back to the MLflow run, logged model, or notebook that produced it — enabling full reproducibility.
Production-Ready Workflows	Aliases (e.g. @champion) and tags support controlled, auditable promotion through experimental, staging, and production.
Governance & Compliance	Structured metadata, tagging, and role-based access control (on managed platforms) support enterprise-grade ML governance.

CORE CONCEPT: MODEL URI

By version	<code>models:/<name>/<version></code>	e.g. <code>models:/MyModel/1</code>
By alias	<code>models:/<name>@<alias></code>	e.g. <code>models:/MyModel@champion</code>

Two ways to register, load by version or alias, and how OSS MLflow compares to Databricks Unity Catalog

```
register_and_load.py

# Option 1 -- register while logging
mlflow.sklearn.log_model(
    ..., registered_model_name="model"
)

# Option 2 -- register an already-logged model
mlflow.register_model(
    f"runs:{best_run_id}/model", "insurance-approval-model")

# Load a specific version
mlflow.sklearn.load_model("models:/model/1")

# Load via alias -- consumer code never changes
mlflow.sklearn.load_model("models:/model@champion")
```

KEY CONCEPT — MODEL ALIAS

A mutable, named pointer to one specific model version.

Reassign the alias to point at a new version to roll out a model — the alias name in downstream code and pipelines never changes.

OSS MLFLOW vs. DATABRICKS UNITY CATALOG

Aspect	OSS MLflow	Databricks Unity Catalog
Registry URI	Local / default tracking store	set_registry_uri("databricks-uc")
Governance	Tags & descriptions	Fine-grained RBAC, access policies
Scope	Single tracking store	Cross-workspace shared catalog
Model signature	Optional	Required

Register the Best Model + @champion Alias

CODE

```
client.get_model_version_by_alias(MODEL_NAME, ALIAS)
```



mlflow3.16.0

GenAIModel training

HomeExperimentsModel registry

Registered Models >

insurance-approval-model

Created Time: 09/19/2026, 08:10:35 PMLast Modified: 09/20/2026, 04:31:54 PM

> Description Edit

> Tags

▼ VersionsCompare

Version	Registered at	Created by	Tags	Aliases
☒ Version 4	09/20/2026, 04:31:54 PM		Add	@ champion✎
☒ Version 3	09/20/2026, 11:53:27 AM		Add	Add
☒ Version 2	09/19/2026, 08:58:07 PM		Add	Add
☐ ☒ Version 1	09/19/2026, 08:10:35 PM		Add	Add

< PreviousNext >25 / page▼

Assignments Experiment Tracking with MLflow

— CODE



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



1. Inspect any fatal issue within the `train_with_mlflow.py` code and explain
2. Improve the code to solve the issue



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



02

Machine Learning Deployment Strategies

10:45–12:00

IN THIS SESSION

- 01 Serving patterns
- 02 FastAPI with a validated contract
- 03 A self-contained container image
- 04 Deployment strategy

Model Serving Patterns

CONCEPT



Pattern	How it works	When to use it
Real-time (REST/gRPC)	Request in → instant prediction → response	A decision needs an immediate answer, e.g. validation at submission time
Batch	Scheduled predictions over a whole dataset	High volume without an instant need, e.g. periodic risk scoring
Streaming	Predictions on a continuous data stream (Kafka, Pub/Sub)	Data flows constantly, e.g. transaction anomaly detection

Today's hands-on uses the real-time (REST) pattern, because that is what we containerise and deploy.

An API Contract with Pydantic

CODE

api/app.py

```
class ApplicantData(BaseModel):
    age: float = Field(..., examples=[34])
    income: float = Field(..., examples=[78000])
    bmi: float = Field(..., examples=[24.5])
    tenure_months: float = Field(..., examples=[18])
    num_claims: float = Field(..., examples=[1])
    credit_score: float = Field(..., examples=[720])
    policy_value: float = Field(..., examples=[150000])
    risk_score: float = Field(..., examples=[42.5])

class PredictionResponse(BaseModel):
    approved: bool
    probability: float
    model_version: str
```

A two-way contract

- Requests are validated before the handler runs
- Missing field or wrong type → HTTP 422 automatically
- response_model keeps the output shape stable
- The /docs page is generated from this schema



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Loading the Model & the /predict Endpoint



CODE

```
api/app.py (abridged)

if os.path.exists(MODEL_PATH):          # container path
    model = joblib.load(MODEL_PATH)      # + model_info.json
else:                                   # local development path
    mv = MlflowClient().get_model_version_by_alias(MODEL_NAME, "champion")
    model = mlflow.sklearn.load_model(f"models:{MODEL_NAME}/{mv.version}")
MODEL_VERSION = f"{type(model).__name__}-v{version}"

@app.post("/predict", response_model=PredictionResponse)
def predict(data: ApplicantData):
    df = pd.DataFrame([data.model_dump()])
    proba = model.predict_proba(df)[0][1]
    return PredictionResponse(
        approved=bool(proba >= 0.5),
        probability=round(float(proba), 4),
        model_version=MODEL_VERSION)
```

Run and Test the API

HANDS-ON 2



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



● ● ● bash – terminal (real output)

```
$ make serve      # PREDICTION_LOG=logs/predictions.jsonl uvicorn api.app:app --reload
```

```
INFO:      Uvicorn running on http://127.0.0.1:8000
```

```
# second terminal
```

```
$ curl http://127.0.0.1:8000/
```

```
{"status":"ok","service":"insurance-approval-api",  
  "model_version":"RandomForestClassifier-v1","model_source":"MLflow Model Registry (@champion)"}
```

```
$ curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" \  
  -d '{"age":34,"income":78000,"bmi":24.5,"tenure_months":18,  
      "num_claims":1,"credit_score":720,"policy_value":150000,"risk_score":42.5}' \  
  {"approved":false,"probability":0.4112,"model_version":"RandomForestClassifier-v1"}
```

Swagger UI — http://127.0.0.1:8000/docs

REAL SCREENSHOT



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Curl

```
curl -X 'POST' \  
  'http://127.0.0.1:8000/predict' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "age": 34,  
    "income": 78000,  
    "bmi": 24.5,  
    "tenure_months": 18,  
    "num_claims": 1,  
    "credit_score": 720,  
    "policy_value": 150000,  
    "risk_score": 42.5  
  }'
```

Request URL

```
http://127.0.0.1:8000/predict
```

Server response

Code	Details
200	Response body<pre>{ "approved": false, "probability": 0.4112, "model_version": "RandomForestClassifier-v1" }</pre> Download

Swagger UI loads its assets from a CDN. If the network blocks it, /docs is blank — use curl or make drill-check instead.

Invalid Requests Are Rejected Automatically

VALIDATION

01 Missing field

A request without `credit_score` → HTTP 422, with no extra validation code.

02 Wrong type

`credit_score = "high"` is rejected before `predict()` ever runs.

03 Structured errors

The 422 body names the field and the reason — easy for client teams to fix.

```
# real response when credit_score is missing
{"detail":[{"type":"missing","loc":["body","credit_score"],"msg":"Field required", ...}]}

# real response when credit_score = "high"
{"detail":[{"type":"float_parsing","loc":["body","credit_score"],
  "msg":"Input should be a valid number, unable to parse string as a number","input":"high"}]}
```



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



From the Registry to a Self-Contained Image

CONTAINERISATION



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



step 0 – export (real output)

```
$ make export
```

```
Model 'insurance-approval-model'  
version 1 (RandomForestClassifier)  
exported to api/models/model.pkl
```

```
# api/models/model_info.json
```

```
{  
  "model_name": "insurance-approval-model",  
  "version": "1",  
  "model_class": "RandomForestClassifier",  
  "run_id": "..."  
}
```

docker/Dockerfile

```
FROM python:3.11-slim  
WORKDIR /app
```

```
# requirements first -> cached pip layer  
COPY api/requirements.txt .  
RUN pip install --no-cache-dir \  
    -r requirements.txt
```

```
COPY api/app.py .  
COPY api/models/ ./models/
```

```
ENV PORT=8080
```

```
# exec: uvicorn is PID 1, receives SIGTERM  
CMD exec uvicorn app:app \  
    --host 0.0.0.0 --port ${PORT}
```

Build & Run the Container Locally

HANDS-ON 3



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



bash — terminal

```
$ make docker-build      # docker build -f docker/Dockerfile -t insurance-api .
$ make docker-run        # docker run -d -p 8080:8080 --name api insurance-api
$ python3 scripts/test_deployment.py http://127.0.0.1:8080
>> Health check...
200 {'status': 'ok', ..., 'model_version': 'RandomForestClassifier-v1',
    'model_source': 'local file (api/models/model.pkl)'}
>> Predict (compare elapsed time for cold vs warm start)...
200 {'approved': False, 'probability': 0.4112, 'model_version': 'RandomForestClassifier-v1'}
$ docker stop api && docker rm api
```

Layer caching

Change one line in app.py and rebuild: the pip install layer stays CACHED.

model_source

Must say "local file" — proof that the image does not depend on MLflow.

Safely Replacing a Model in Production

RELEASE STRATEGIES



Strategy	How it works	Strength	Risk / cost
Recreate	Stop the old version, start the new one	Simplest	Downtime; slow rollback
Blue/Green	Two identical environments; switch traffic at once	Instant rollback	Double capacity
Canary	The new version gets a small share of traffic (e.g. 10%)	Limited blast radius	Needs per-version monitoring
Shadow	The new version gets a copy of traffic; its output is ignored	Zero user risk	Double compute cost
A/B test	Traffic split at random; impact compared statistically	Measures business impact	Needs volume & experiment design

03

Monitoring and Observability

13:15–14:45

IN THIS SESSION

01 Monitoring vs observability

02 Structured prediction logs

03 Data drift with Evidently AI

04 Reading the drift report

05 Retraining policy

Monitoring vs Observability

CONCEPT



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Monitoring

Answers: is the system healthy?

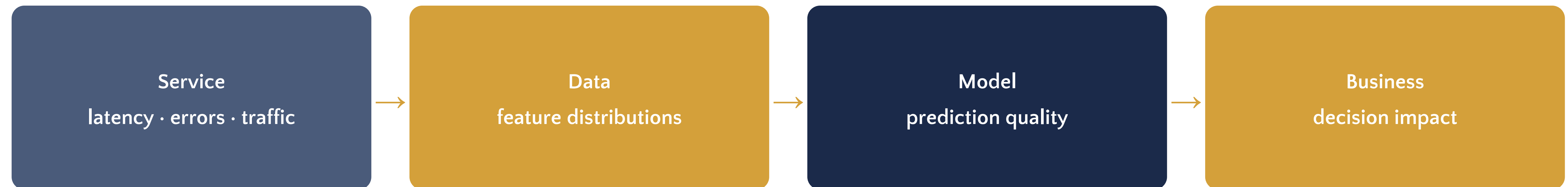
- Predefined metrics and thresholds
- Alerts when a threshold is crossed
- Example: p95 latency > 500 ms, drift > 30%

Observability

Answers: why is it behaving this way?

- Context-rich logs, metrics and traces
- Lets you investigate problems you did not predict
- Example: per-prediction logs with model version & features

What to watch in an ML system



Evidently Major Monitoring Capabilities

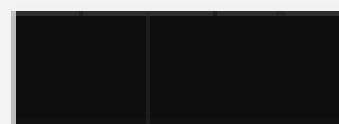
TAXONOMY



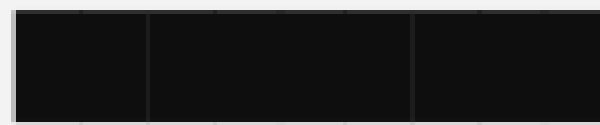
No	Area	What it answers?
1	Data Quality	Is the incoming data valid? → Is there any missing data, invalid data, invalid format?
2	Data Drift	Has input distribution changed?
3	Prediction Drift	Has model output changed?
4	Model Performance	Is the model still accurate?
5	Model Quality	Is the model behaving acceptably according to domain metrics? → Define threshold requirements

age:

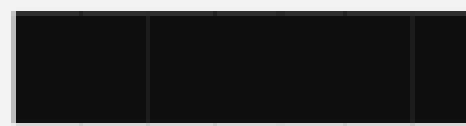
18–25



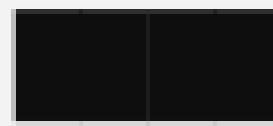
26–35



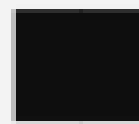
36–45



46–55



56+

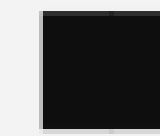


Data for training and evaluation

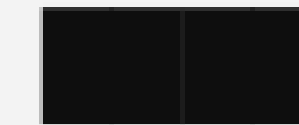
vs.

age:

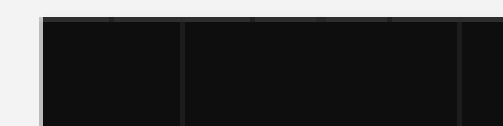
18–25



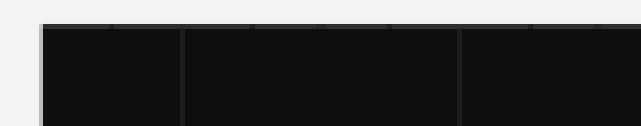
26–35



36–45



46–55



56+



Data production

Evidently can compare: Reference dataset VS Current production dataset

Reference period

Approved 70%

Rejected 30%

Testing results during development

vs.

Current period

Approved 92%

Rejected 8%

Prediction results in production

Evidently can monitor the prediction results in production and compared with reference values populated during model development

Evidently monitors model performance based on pre-defined metrics

Classification	Regression
Accuracy Precision Recall F1 ROC-AUC confusion matrix calibration-related measures	MAE ME RMSE error distributions error normality error bias

The prediction in production is compared with the ground truth to measure model performance

Evidently API Summary

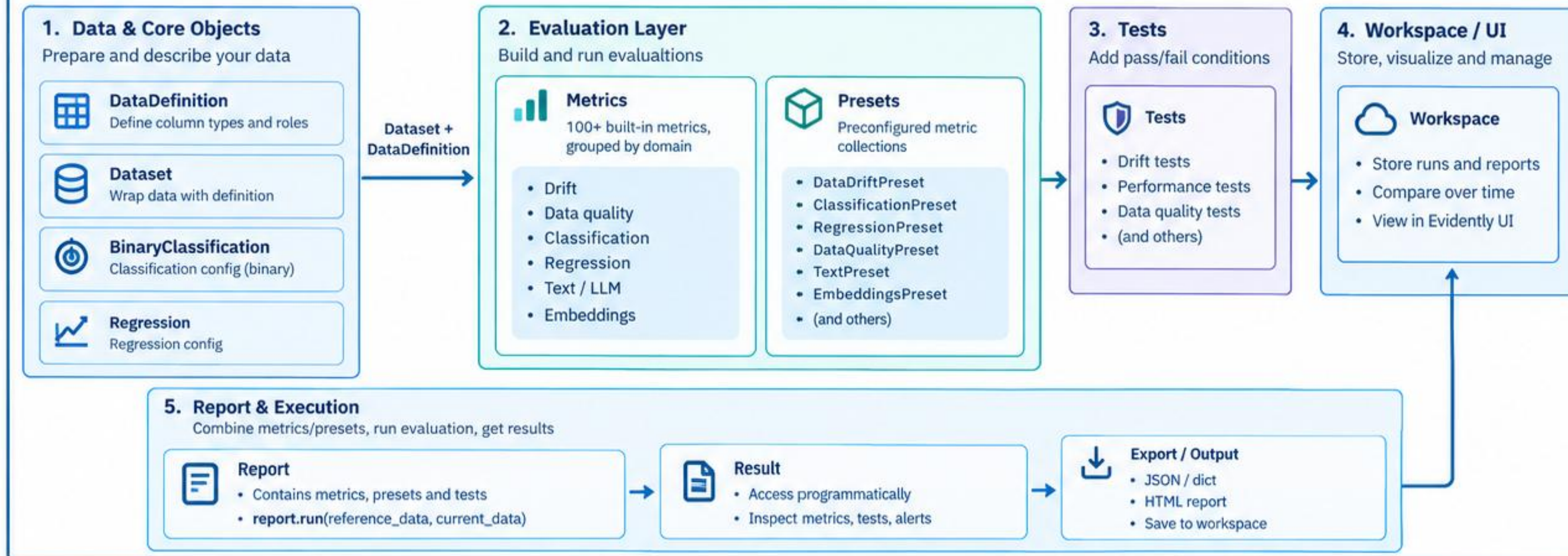
TAXONOMY



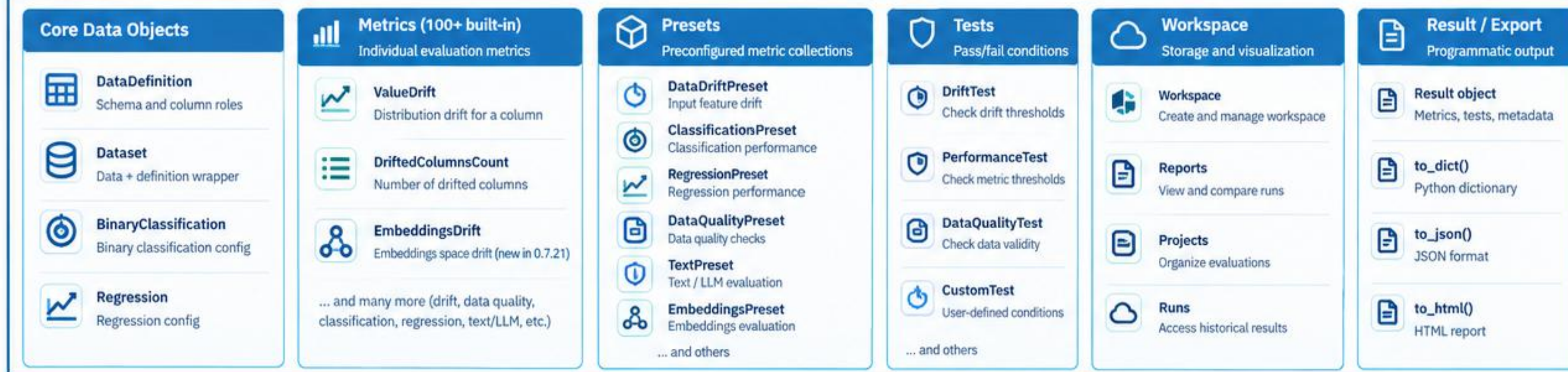
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



1. High-Level API Architecture



2. Detailed API Components



Example API Usage

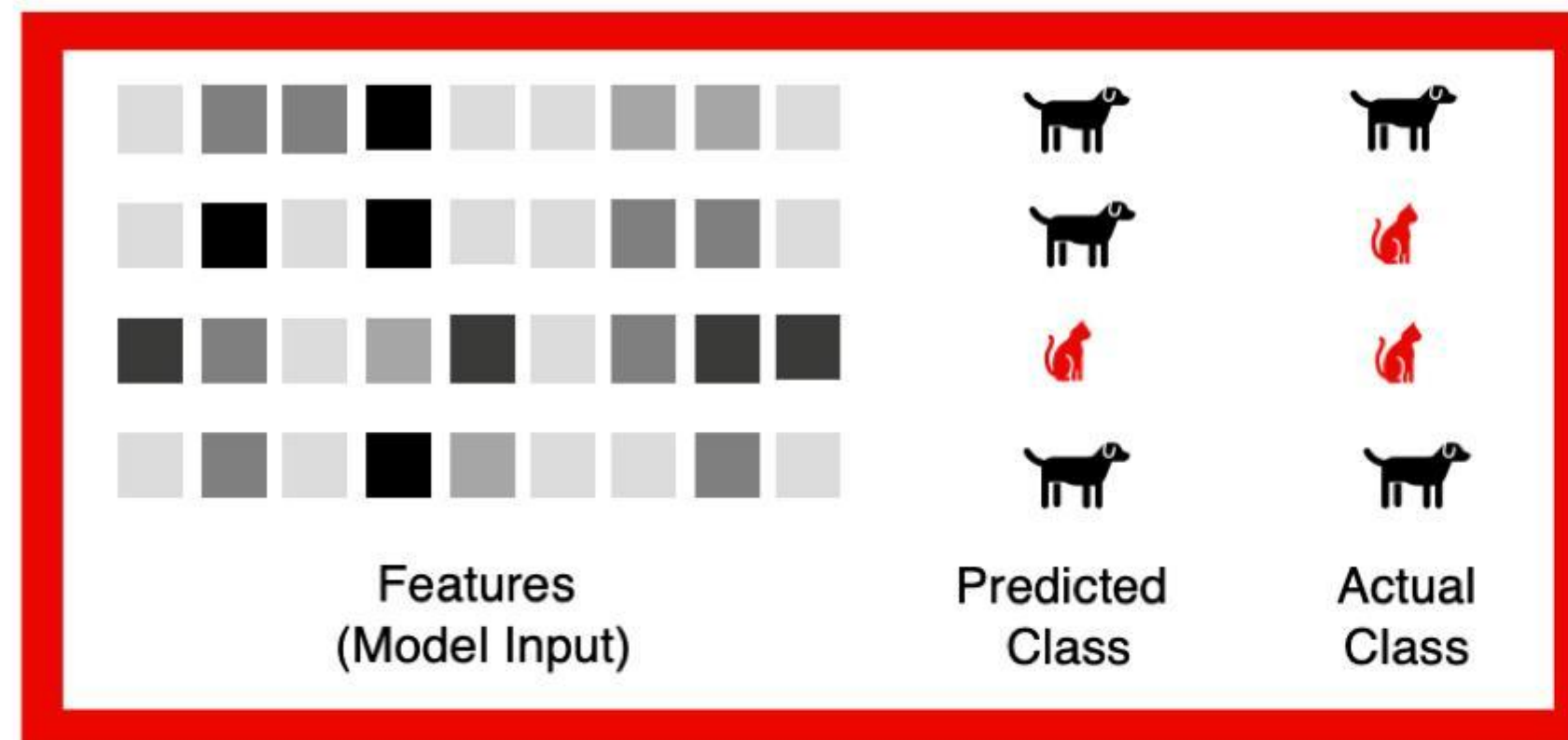
TAXONOMY



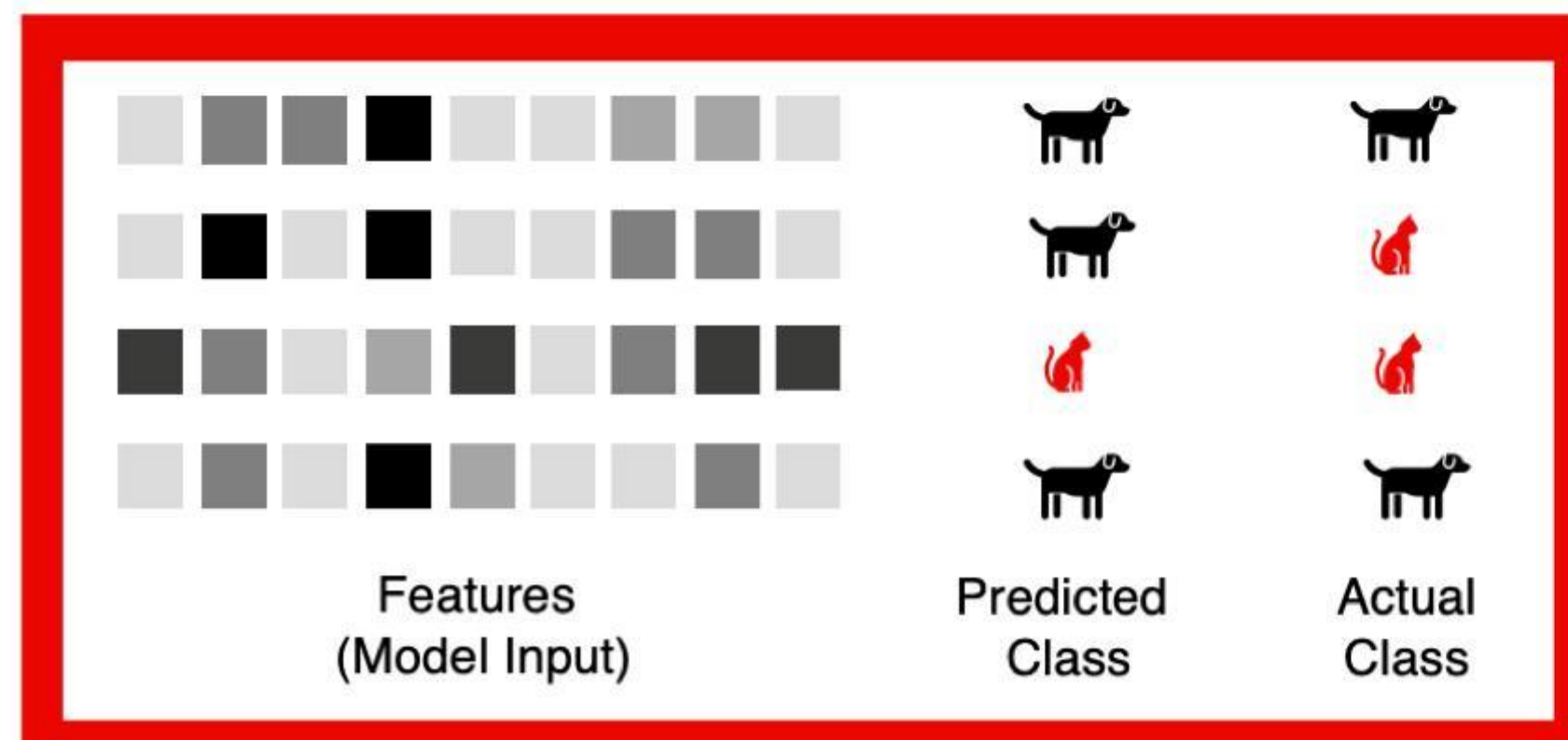
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Reference Dataset

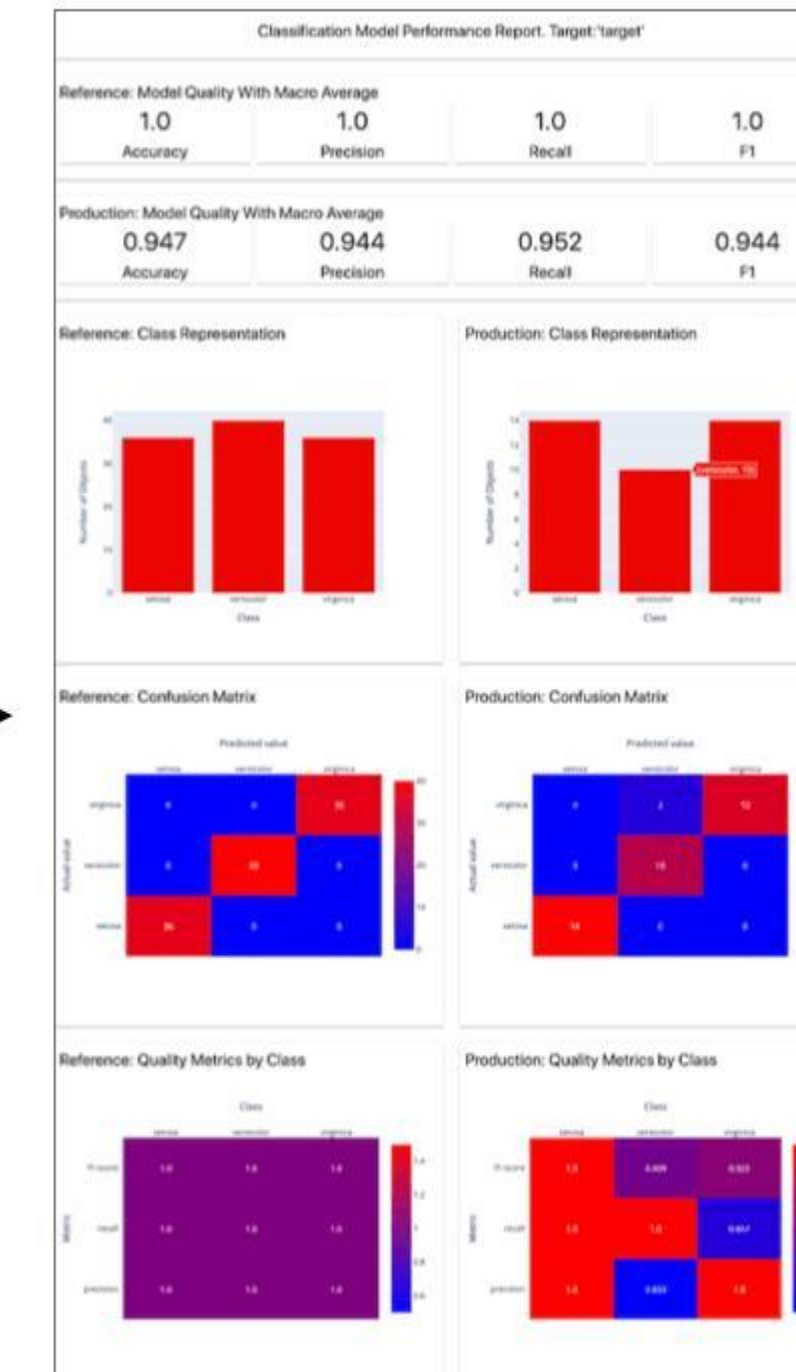


Current Dataset



```
report = Report(metrics=[  
    ClassificationPreset(),  
])  
  
report.run(reference_data=ref,  
current_data=cur)  
  
report
```

Performance Report

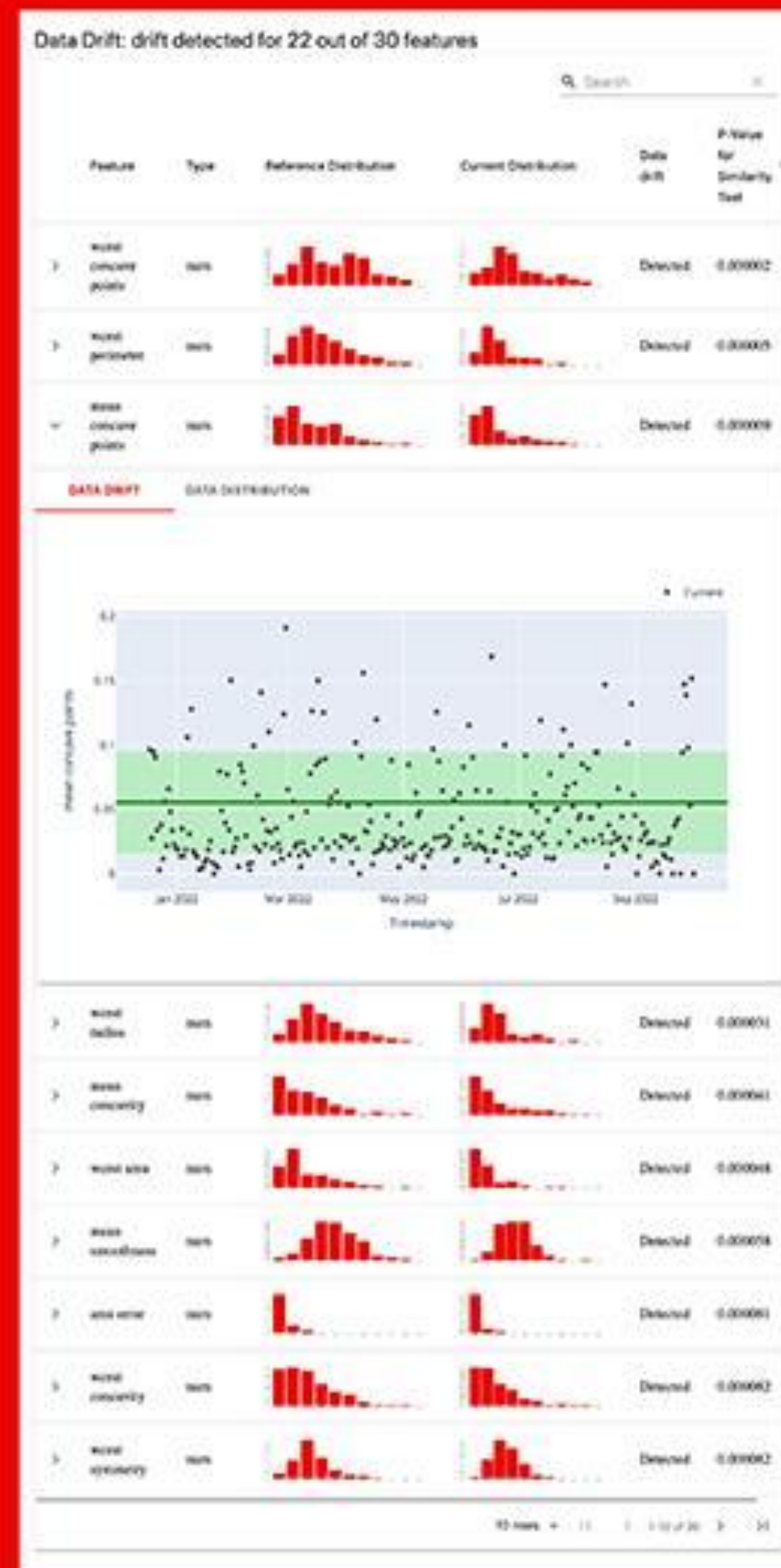


Example Evidently Report

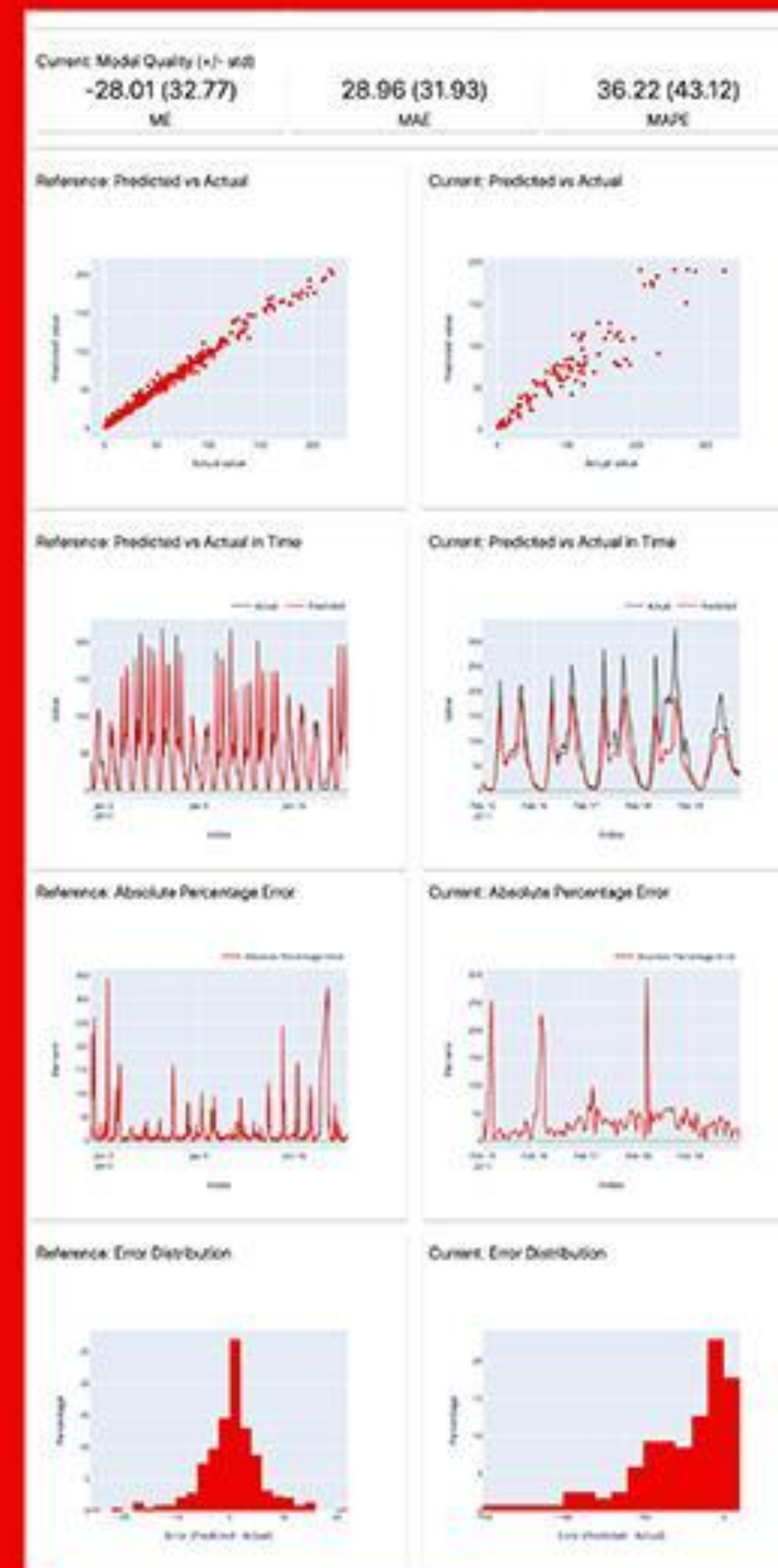
TAXONOMY



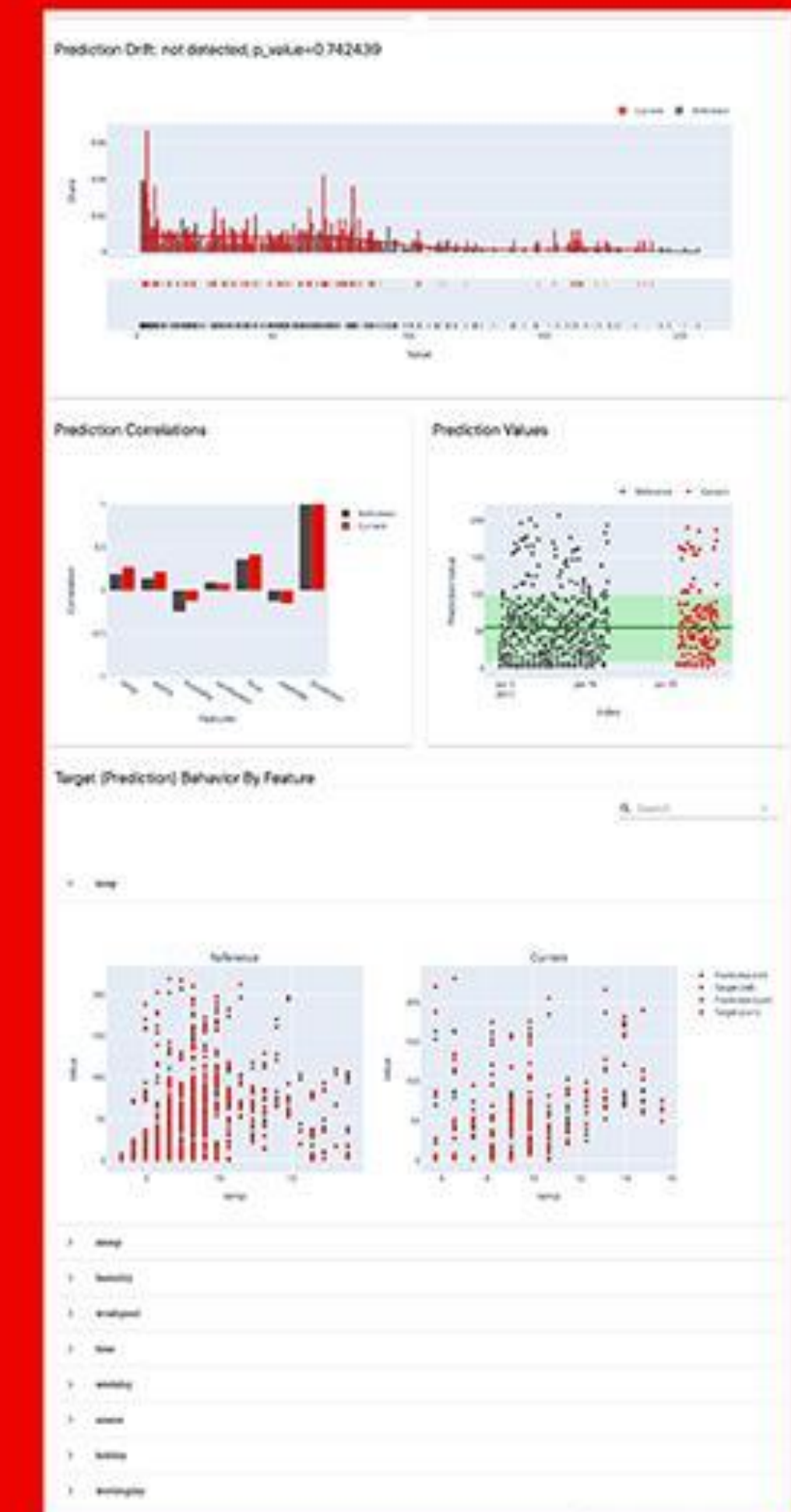
DATA DRIFT



MODEL PERFORMANCE



TARGET DRIFT



Evidently Sequence for Running Monitoring

OBSERVABILITY



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



```
43 from evidently import Report, Dataset, DataDefinition, BinaryClassification
44 from evidently.presets import DataDriftPreset, ClassificationPreset
45 from evidently.metrics import ValueDrift
46 from evidently.ui.workspace import Workspace
47
```

```
65 # Reference: representasi data yang dipakai saat training/evaluasi (Sesi 2-4)
66 reference = pd.read_csv("data/test.csv").copy()
67
68 # Current: data produksi -- idealnya log request dari endpoint lokal (Sesi 6),
69 # bukan file statis seperti contoh ini.
70 current = pd.read_csv("data/production.csv").copy()
```

```
report = Report([
    DataDriftPreset(columns=FEATURES),      # sinyal 1 -- HANYA 8 fitur input
    ValueDrift(column="prediction"),        # sinyal 2 -- output model
    ClassificationPreset(),                 # sinyal 3 -- performa CURRENT
])
snapshot = report.run(current_data=current_ds, reference_data=reference_ds)
```

Calculating Drift

CODE

```
def _metric_value(snap, metric_type: str, column: str = None):
    """Ambil value metric dari snapshot. Jika `column` diisi, filter juga
    berdasarkan config['column'] -- DataDriftPreset menghasilkan SATU
    ValueDrift per fitur (age, income, dst.), jadi mencari type saja tidak
    cukup untuk mengambil ValueDrift milik kolom 'prediction' secara spesifik."""
    for m in snap.dict()["metrics"]:
        if m["config"]["type"] != metric_type:
            continue
        if column is not None and m["config"].get("column") != column:
            continue
        return m["value"]
    return None

n_drifted_share = _metric_value(snapshot, "evidently:metric_v2:DriftedColumnsCount")
prediction_drift_pvalue = _metric_value(snapshot, "evidently:metric_v2:ValueDrift", column="prediction")
cur_acc = _metric_value(snapshot, "evidently:metric_v2:Accuracy")
cur_auc = _metric_value(snapshot, "evidently:metric_v2:RocAuc")
ref_acc = _metric_value(ref_snapshot, "evidently:metric_v2:Accuracy")
ref_auc = _metric_value(ref_snapshot, "evidently:metric_v2:RocAuc")

n_drifted, n_total = int(n_drifted_share["count"]), len(FEATURES)
data_drift_share = n_drifted_share["share"]
prediction_drift_detected = prediction_drift_pvalue < 0.05
```



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Storing Report to Local vs Workspace

HANDS-ON 4



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



```
os.makedirs(os.path.dirname(os.path.join("monitoring", "drift_report.html")), exist_ok=True)
snapshot.save_html("monitoring/drift_report.html")
print("Laporan mandiri tersimpan di monitoring/drift_report.html")
```

```
# --- Simpan ke Evidently Workspace agar terlihat di Evidently UI -----
workspace = Workspace.create(WORKSPACE_PATH)
existing = workspace.search_project(PROJECT_NAME)
project = existing[0] if existing else workspace.create_project(
    PROJECT_NAME, description="Monitoring ML model using evidently."
)
workspace.add_run(project.id, snapshot, include_data=False)
print(f"\nSnapshot tersimpan ke workspace '{WORKSPACE_PATH}' (project: {PROJECT_NAME})")
print("Buka via Evidently UI: docker compose -f docker/docker-compose.yml up -d evidently-ui")
print("    lalu akses http://localhost:9002")
```

The Evidently Drift Report

— REAL SCREENSHOT

Check drift_report.html or execute run_evidently_service.sh → localhost:9002

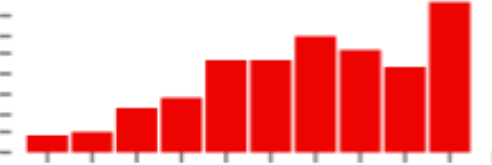
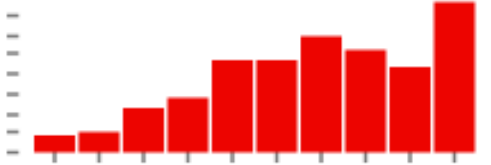
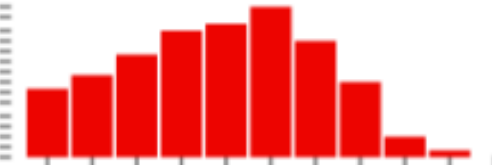
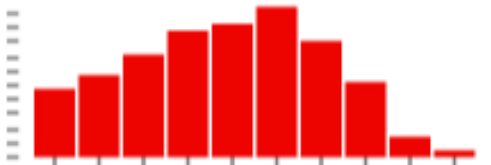
Dataset Drift

Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

8
Columns

3
Drifted Columns

0.375
Share of Drifted Columns

	Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
>	credit_score	num			Detected	K-S p_value	0
>	income	num			Detected	K-S p_value	0
>	num_claims	num			Detected	K-S p_value	0

The Evidently Drift Report

— REAL SCREENSHOT

Check drift_report.html or execute run_evidently_service.sh → localhost:9002



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Classification Model Performance. Target: 'approved'

Current: Model Quality Metrics

0.888

Accuracy

0.912

Precision

0.852

Recall

0.881

F1

0.958

ROC AUC

0.324

LogLoss

Reference: Model Quality Metrics

0.938

Accuracy

0.957

Precision

0.914

Recall

0.935

F1

0.982

ROC AUC

0.245

LogLoss

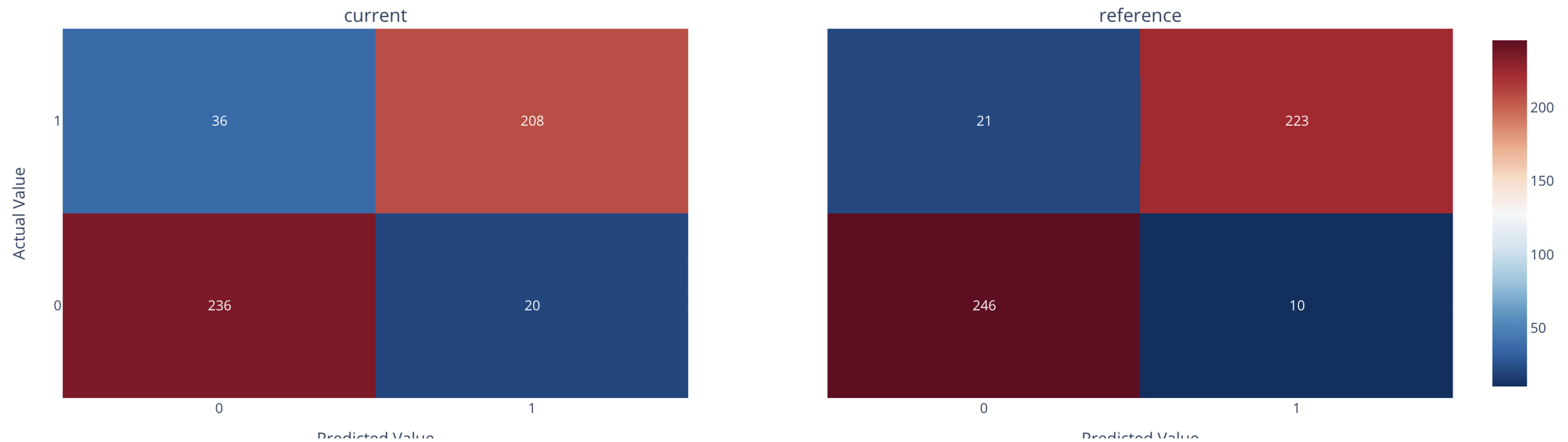
The Evidently Drift Report

— REAL SCREENSHOT

Check `drift_report.html` or execute `run_evidently_service.sh` → `localhost:9002`



Confusion Matrix



The Evidently Drift Report

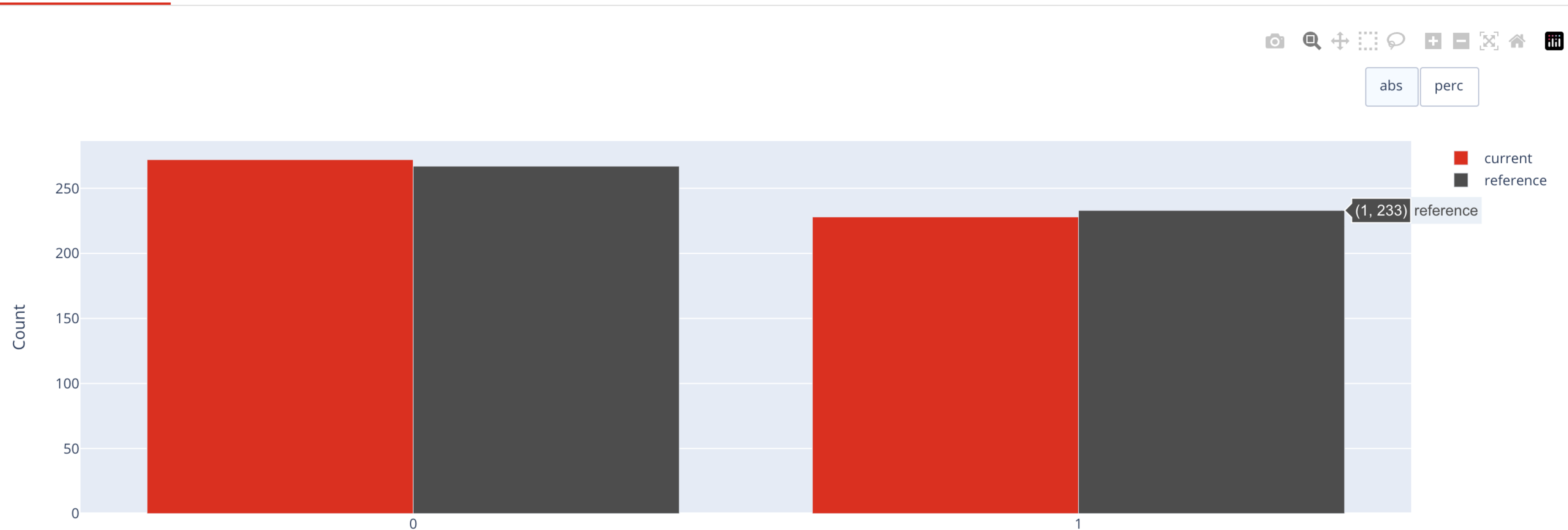
REAL SCREENSHOT

Check `drift_report.html` or execute `run_evidently_service.sh` → `localhost:9002`

Drift in column 'prediction'

Data drift not detected. Drift detection method: Z-test p_value. Drift score: 0.751

DATA DISTRIBUTION



01 3 of 8 columns drifted

income, num_claims, credit_score — K-S p-value = 0,
exactly the three features that were shifted.

02 Share $0.375 < 0.5$

Below Evidently's default threshold, so the overall status
reads "Dataset Drift NOT detected".

03 5 stable columns

age, bmi, policy_value, risk_score, tenure_months were
not shifted — p-value = 1.

04 Implication

The aggregate status can mislead. A retraining policy
should also watch key features individually.

Defining a Retraining Policy

— OPERATING POLICY



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



- 01 Set a threshold, e.g. retrain if $\geq 30\%$ of columns drift for 3 consecutive days
- 02 Run orchestration/pipeline.py daily (a Prefect schedule or Cloud Run Jobs)
- 03 Store `report.as_dict()` in a table to see trends, not just snapshots
- 04 When the threshold is crossed, trigger the training pipeline (Session 1) on fresh data
- 05 The new model becomes @challenger → passes the quality gate → canary → @champion

Our Simple Policy

HANDS-ON 4



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



```
162 accuracy_drop = ref_acc - cur_acc
163 drift_trigger = data_drift_share >= DRIFT_THRESHOLD
164 performance_trigger = accuracy_drop >= ACCURACY_DROP_THRESHOLD
165
166 print()
167 if drift_trigger or performance_trigger:
168     alasan = []
169     if drift_trigger:
170         alasan.append(f"data drift {data_drift_share:.2f} >= ambang {DRIFT_THRESHOLD}")
171     if performance_trigger:
172         alasan.append(f"accuracy turun {accuracy_drop:.4f} >= ambang {ACCURACY_DROP_THRESHOLD}")
173     print(f"⚠ Kebijakan retraining melewati ambang ({'; '.join(alasan)}) -- pertimbangkan retraining.")
174 else:
175     print("Kedua sinyal masih di bawah ambang -- belum perlu retraining.")
176
```

We will use this result for defining automation flow with prefect



04

Orchestration

15:00–15:40

IN THIS SESSION

01 Prefect configuration and running server

02 Defining prefect flow

03 Create automation with prefect

04 Exercise and discussion

When Should a Model Be Retrained?

CONTINUOUS TRAINING



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



01 On a schedule

Retrain daily, weekly or monthly. Simple, but wasteful when the data has not changed.

02 New data arrives

Retrain when a new batch of labelled data lands, e.g. at every period close.

03 Drift detected

Retrain when monitoring shows feature distributions shifting past a threshold (Session 3).

04 Performance drops

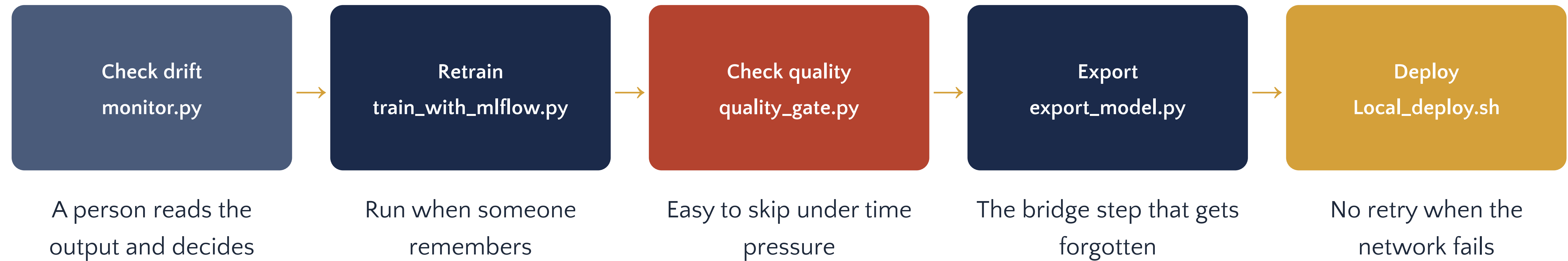
Retrain when live metrics (with ground truth) fall below the agreed threshold.

Whatever the trigger, a retrained model must pass the same quality gate before it becomes @champion.

Manual Orchestration Is Maturity Level 0



PROBLEM



Each arrow is a human decision, not code. The retraining policy exists only as a sentence in a document.

Orchestration moves the arrows into code: order, conditions, retries and history.

What an Orchestrator Adds

CONCEPT



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



01 Scheduling

The pipeline runs every night at 02:00, not whenever an engineer remembers to run it.

02 Conditional branching

"Release ONLY IF the candidate passes the gate" — logic that used to live in someone's head.

03 Closing the loop

The retraining policy from Session 3 ("retrain if $\geq 30\%$ of columns drift") is actually executed.

04 Retries & history

A flaky gcloud call is retried automatically; every run is recorded and can be audited in a UI.

Orchestration Tools: Infrastructure Trade-offs

COMPARISON



Tool	Execution model	Infrastructure required	When to choose it
Airflow	Scheduler + static DAGs	Metadata DB (Postgres), webserver, scheduler	Many non-ML pipelines already; one central scheduler
Kubeflow Pipelines	One container per step on K8s	A Kubernetes cluster	Steps need large resources (GPUs) each
Prefect	Native Python, dynamic	None to start — @flow is a Python function	Small-to-medium teams; no extra servers
ZenML	Abstraction over other tools	Local SQLite for the basics	Gradual migration reusing existing MLflow/storage

We use Prefect today because it is a plain pip install. The design ideas carry over to Airflow or Kubeflow unchanged.

A Prefect Flow with a Quality Gate

CODE

```
orchestration/pipeline.py (abridged)

@task(retries=2, retry_delay_seconds=5, log_prints=True)
def check_drift() -> bool:
    return "exceeds threshold" in _run([PY, "monitoring/monitor.py"])

@task(retries=3, retry_delay_seconds=10, log_prints=True)
def deploy():
    _run(["bash", "docker/deploy.sh"])

@flow(name="insurance-approval-retraining", log_prints=True)
def mlops_pipeline():
    if not check_drift():
        return # nothing to do
    previous = _champion_version()
    retrain()
    if not quality_gate():
        rollback(previous) # never release a failing candidate
        return
    export_model()
    if os.getenv("PROJECT_ID"):
        deploy()

if __name__ == "__main__":
    mlops_pipeline.serve(
        name="insurance-approval-test",
        interval=60,
    )
```

What Prefect adds

- @task: retries, timing and logs per step
- @flow: the order and the if-branches live in code
- _run() shows a failing step's stderr instead of hiding it
- Deploy runs only when PROJECT_ID is set
- In this snippet the monitoring will be executed once every 60 seconds. Note that prefect process can be placed as Cron job.



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



05

Incident Handling

15:40–16:00

IN THIS SESSION

01 Types of ML incidents

02 Contoh Kategori Insiden MLOps

03 Level Kritikalitas Insiden

Types of Incidents in ML Systems

CONCEPT



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



01 Model quality

The model makes wrong predictions at scale: a bad release or sudden concept drift.

02 Data pipeline

Wrong joins, swapped columns, shuffled labels, late data — the model is trained on broken data.

03 Service & infrastructure

Endpoint down, latency spikes, repeated cold starts, exhausted cloud quota.

04 Security & privacy

Sensitive data leaking through logs, unauthorised access, attacks on the model (MLSecOps, Day 1).

Typical ML related incidents

CONCEPT



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



➤ Tidak ada crash, tidak ada error code

Endpoint tetap mengembalikan HTTP 200, namun jawabannya yang perlahan menjadi tidak akurat, server tetap berjalan

➤ Baseline yang bergeser tanpa terdeteksi

Perubahan pada lingkungan operasi (musiman, perilaku pengguna, kondisi ekonomi), asumsi training tidak lagi berlaku, dimana perubahan ini tidak terprediksi

➤ Monitoring tradisional gagal menangkapnya

CPU, latency, error rate semua tampak normal, sehingga dibutuhkan instrumentasi khusus untuk melihat penurunan yang sesungguhnya

➤ Pola silent failure adalah hal normal

Prediction drift TIDAK terdeteksi, padahal nilai accuracy model telah turun 5%.

Contoh Kategori Insiden MLOps

CONCEPT



Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



Kategori	Phase	Contoh Insiden	Dampak
Data & Model	Model Development	Data drift, prediction drift diam-diam, alias registry menunjuk versi salah	Keputusan bisnis salah tanpa disadari
Serving Infrastructure	Model Deployment	API crash, breaking change dependency, container gagal start	Layanan tidak bisa diakses
Pipeline & Orchestration	Orkestrasi	Model buruk ter-promosi otomatis, task gagal diam-diam, retry exhaust	Otomasi mempercepat kesalahan, bukan cuma layanan
Monitoring Blind Spot	Model Monitoring	Satu sinyal saja dipasang, threshold salah kalibrasi, alert fatigue	Insiden nyata luput terdeteksi sama sekali



Level	Kriteria	Contoh pada materi Workshop	Target Response
Kritis	Layanan down / keputusan salah massal	insurance-api tidak merespons, atau meloloskan/menolak semua permohonan	Segera, hentikan pekerjaan lain
Tinggi	Performa turun signifikan, terkonfirmasi	Accuracy turun ≥ 0.03 terkonfirmasi ground truth	< 1 jam
Sedang	Drift terdeteksi, masih dalam batas aman	Data drift share < 0.30, belum memicu kebijakan retraining	Investigasi terjadwal
Rendah	Anomali kecil, sifatnya informasional	Satu fitur bergeser tapi tidak signifikan secara statistik	Catat, review berkala

Tools Model Registry dan Monitoring membantu kita dalam menghadapi insiden. Melalui monitoring berkelanjutan silent failure dapat dideteksi lebih awal. Model registry mendukung proses rollback yang cepat.

Thank you!

Any question?
